RoboCup Soccer Simulation: Symbolic AI in Prolog

Team members:

Chayut Lohachitranond	*6738048421*
Dimitrios Iliadis	*6802024421*
Elena-Alexandra Ciobotaru	*6802027529*
Aziza Fourati	*6802019321*

1. Introduction

RoboCup Soccer Simulation is an environment where two teams of autonomous agents compete on a discrete field using a defined set of actions: move, kick, catch, and tackle. This project implements a core symbolic AI engine for simulating a simplified version of that; two teams of three players each (goalkeeper, defender, forward) on a 100×50 grid, using three symbolic techniques as the primary reasoning layers: Constraint Satisfaction Programming (CSP), Finite State Machines (FSM), and STRIPS action schemas. Sub-symbolic approaches such as neural networks or case-based reasoning are out of scope.

The three techniques are layered with no overlap in responsibility: CSP handles initial formation, FSM drives per-player intent selection each tick, and STRIPS enforces action legality and mutates world state. Uncertainty is present by design. Kick outcomes carry a random 0–3 unit shortfall, a tackle success is a 50% probabilistic roll, and team order is randomised per each round. These are deliberate design choices to reflect that real soccer involves imperfect execution; the symbolic reasoning layer remains deterministic while action outcomes are uncertain.

2. Symbolic Representations

2.1 Knowledge Base

The system's knowledge splits into two tiers. **Static knowledge** encodes domain axioms: field dimensions, goal positions, action ranges, stamina costs, and stochastic parameters (kick shortfall bound, tackle success rate). **Dynamic knowledge** holds the current world state: ball position, player positions and stamina, possession, FSM intent states, score, and performance counters. Static facts encode *what the domain is*; dynamic facts encode *what is currently true*. Dynamic facts are updated atomically as game events occur.

2.2 Layered Architecture

The three techniques are stacked as independent layers, each with a single point of entry:

Layer	Technique	Responsibility
Setup	CSP	Compute a valid initial formation before the match
Intent	FSM	Select each player's behavioural goal each tick
Action	STRIPS	Execute the chosen action legally; update world state

CSP runs once; FSM runs every tick per player; STRIPS executes what FSM decided. Layers communicate only through shared world state, making each independently inspectable and replaceable. (See *System Overview* and *Turn Flow* in diagrams/overview.svg, diagrams/turnflow.svg)

3. Strategies and Rationale

3.1 Constraint Satisfaction Programming; Initial Formation

Six integer variables encode player coordinates (X, Y) for goalkeeper, defender, and forward. Unary domain constraints bound each role to its initial placement zone; three binary constraints require every pair of teammates to be at least 15 grid units apart in Manhattan distance:

$$|X_i - X_j| + |Y_i - Y_j| \geq 15 \quad \forall i \neq j \in \{gk, df, fw\}$$

Role	X domain	Y domain	Rationale
Goalkeeper	[1, 15]	[20, 30]	Near own goal mouth
Defender	[5, 35]	[10, 40]	Defensive third, own half
Forward	[40, 50]	[10, 40]	Near midline, own half

Team2 domains are mirrored about X = 50; cross-team spacing is structurally guaranteed without

additional constraints. CSP was chosen over manual coordinates because the solver guarantees formation legality with no risk of players clumping, while providing randomness in initial position.

CSP runs exactly once at match setup; re-solving per round would disrupt ongoing play. Labeling uses the default heuristic with `once/1` to take the first valid solution and stop, keeping setup deterministic. The spacing threshold of 15 was chosen deliberately to spread the team, small enough that solutions are found immediately. A known gap: between goals players drift freely; CSP only resets on a goal. (See *CSP Constraint Network* in diagrams/csp_initialpos.svg)

3.2 Finite State Machines; Role Intent Selection

Each role has three named intent states; nine transitions total across both teams. Conditions are evaluated by side-effect-free sensor predicates that query world state without modifying it.

Role	From	Condition	To
goalkeeper	*guard_goal*	ball in own half ∧ ball not held	*chase_ball*
goalkeeper	*chase_ball*	within catch range of ball	*hold_ball*
goalkeeper	*hold_ball*	possession lost	*guard_goal*
defender	*hold_line*	ball in own half ∧ ball loose	*intercept*
defender	*intercept*	has possession	*pass_to_forward*
defender	*pass_to_forward*	possession lost	*hold_line*
forward	*advance*	ball loose ∧ no possession	*chase_ball*
forward	*chase_ball*	within shooting range of opponent goal	*shoot*
forward	*shoot*	possession lost	*advance*

FSM was chosen for intent selection because behaviour decomposes cleanly into named modes; each role's strategy is readable as a cycle through the table. Transitions are performed deterministically via `once/1`, this ensures that the first matching rule is always chosen, preventing ambiguity. If no transition, the player holds its current state, which is the correct default (e.g. a goalkeeper already at `guard_goal` when nothing is happening).

The FSM selects *intent only* and never touches the world directly; this strict separation means FSM transitions can be read and tested independently of STRIPS. (See *FSM State Diagrams* in diagrams/fsm/)

3.3 STRIPS; Action Execution

Six actions cover all meaningful in-game events.

Action	Key preconditions	Key effects
Move	Player at pos; stamina ≥ cost; target in field	Position updated; ball follows if carried; stamina −= 5
Kick	Has possession; stamina ≥ cost; target ≤ kick_range	Ball → target + ε (random shortfall ε ∈ [0,3]); possession → none; stamina −= 10
Catch	Actor is goalkeeper; ball loose; ball within catch_range	Ball snaps to GK; possession → GK
Collect	Actor is field player; ball loose; distance ≤ move_step	Ball snaps to player; possession → player
Tackle	Opponent has possession; different teams; distance ≤ move_step	p = 0.5: possession transfers to tackler; 1−p: no change
Pass	Has possession; teammate in kick_range; stamina ≥ cost	Ball → teammate; possession → teammate; stamina −= 10

Preconditions are evaluated directly against the live dynamic world state rather than a separate literal set; equivalent in a single-threaded simulation and simpler to inspect. The `world` argument is retained in the `applicable` predicate signature for formal fidelity.

Uncertainty in action outcomes. Kick carries a random shortfall $\varepsilon \sim \text{Uniform}(0, 3)$ grid units, reflecting imperfect execution; a kick aimed at the goal mouth may land short. Tackle uses a Bernoulli trial with $p = 0.5$, reflecting the contested nature of possession challenges. Both parameters live in the static knowledge base so they can be tuned without touching the reasoning layer. Turn order is also randomised each round, meaning neither team has a structural advantage beyond the first kickoff.

4. Evaluation

4.1 Performance Metrics

Seven counters track in-game activity per team: shots, passes, saves, collects, tackles won/lost, and goals; accumulated across the full match and printed at the end of each run. They give a coarse signal on whether each symbolic layer is active; saves confirm FSM routes the goalkeeper correctly; passes confirm the defender-to-forward pipeline fires; a high tackles-lost rate against low tackles-won signals miscalibrated proximity sensing.

4.2 Unit tests

16 PLUnit tests cover initial world state, CSP spacing guarantees, stamina depletion, STRIPS precondition blocking, goal detection, FSM initial states, and end-to-end termination; all pass. Key guarantees verified: a player without possession cannot kick; a forward cannot catch (role-restricted precondition); CSP always produces pairwise spacing ≥ 15; a ball crossing a goal line always increments the correct team's score.

4.3 Strengths

- **Symbolic legibility:** every game event traces to a named FSM state or STRIPS precondition; no implicit side effects.
- **Technique isolation:** each layer occupies its own code section; changing one does not touch the others.
- **Tuneable constants:** all domain parameters (ranges, costs, stamina, stochastic bounds) live in the static knowledge base; behaviour adjusts without editing any reasoning logic.

4.4 Weaknesses

- **Non-reproducible outcomes:** turn order randomised each round → results vary between runs without a fixed seed.
- **Manhattan approximation:** proximity checks use $|dx| + |dy|$; diagonal distances overestimated → occasional missed interceptions near corners.
- **Stamina exhaustion:** in long runs (~30+ rounds) some players exhaust and the simulation stalls under preconfiged situation.

5. Limitations and Future Work

Improved sensing. Sensor predicates read world state with perfect accuracy; real agents receive noisy partial observations. A configurable noise layer would require FSM transitions to tolerate uncertainty into probabilistic guards or a belief-state layer.

No lookahead. FSM picks one greedy action per tick with no forward search. STRIPS operators already provide the precondition and effect structure for a planner; replacing FSM intent selection with BFS or A* over those operators would require minimal structural change.

Fixed roles and team size. Roles are fixed atoms; scaling to larger team size requires extending the FSM table and CSP domains. Larger variable sets would benefit from a first-fail labeling heuristic.

Static mid-match formation. CSP resets on goals but formations drift freely between them. A partial re-solve triggered when a player exceeds its zone boundary would maintain tactical shape throughout a match.

Team1 kickoff advantage. The opening kickoff is always awarded to team1. One call to `random_member/2` (already used for turn order) would remove the asymmetry.